Информационные технологии

Лекции за 4 семестр

Боже, спаси и сохрани

2026

Содержание

1 ЛК 1: Стиль программирования. Проектирование ПО (1) **2**

2 ЛК 2: Проектироание ПО (2) **3**

3 ЛК 3: Кодирование ПО (1) **4**

4 ЛК 4: Кодирование ПО (2). Документирование ПО **5**

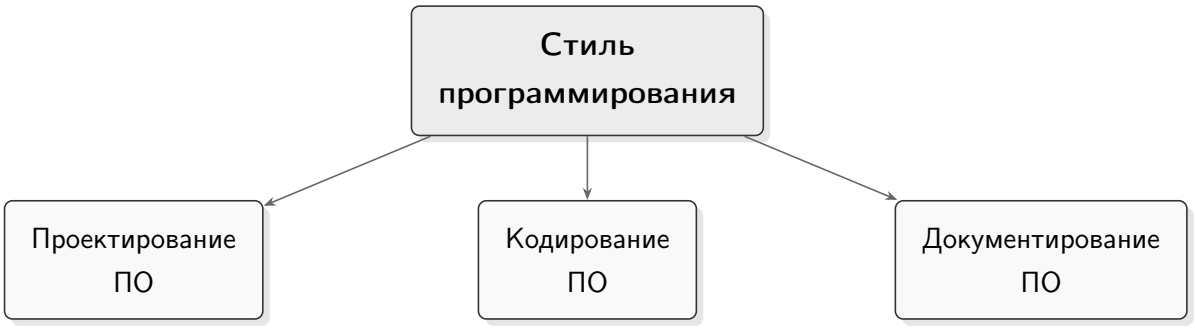
5 ЛК 5: Документирование ПО (2). Советы для C/C++ **7**

6 ЛК 6: Разговоры о разном (1) **9**

7 ЛК 7: Разговоры о разном (2) **11**

1 ЛК 1: Стиль программирования. Проектирование ПО (1)

Стиль программирования



Если имеется несколько способов выполнить действия и выбор осуществляется произвольно, следует придерживаться выбранного варианта. **Главное правило:** не следует изобретать собственный стиль там, где есть общепринятый.

Проектирование ПО (2)

1. **Принцип «без сюрпризов»:** программа не должна удивлять пользователя неожиданным поведением.
2. **Декомпозиция:** разбиение крупных задач на более мелкие и управляемые.
3. **Минимизация связей:** компоненты должны быть максимально независимы друг от друга.
4. **Связность компонентов:** объединение элементов, имеющих общее назначение (интерфейс, логика и т.д.).
5. **Избегание неоправданной сложности (KISS — “Keep It Simple, Stupid):**
   (a) Не стоит пытаться учесть все гипотетические случаи — лучше обеспечить возможность легкого расширения.
   (b) Не изобретать велосипед (не реализовывать то, что уже встроено в язык).

2 ЛК 2: Проектироание ПО (2)

Продолжение ЛК 1

6. Следует думать о потенциальных нуждах пользователя и его удобстве.
7. Если ошибка не критична, нужно дать пользователю шанс ее исправить.
8. Сообщение об ошибке должно быть конструктивным (подсказывать решение).
9. Не сообщать об ошибке, если программа исправила её автоматически (например, автокапитализация).

   Мнение лектора: Барчев назвал этот совет спорным.

10. Диагностика ошибки должна быть максимально приближена к месту её возникновения.
11. Следует внимательно читать код своих коллег с целью поиска и устранения ошибок, понятности кода и дальнейшего его сопрововждения.
12. Избегайте правок чужого кода без веских оснований.
13. Используйте возможности своего инструментария (IDE, компилятор) на 100%.
14. Тщательное обдумывание задачи ускоряет написание и облегчает сопровождение.
15. Вовлекайте заказчика в процесс проектирования.
16. Не используйте неизвестные технологии, если есть проверенные альтернативы с меньшим риском.
17. Не навязывайте пользователю жесткую технологию работы.
18. Качество проекта важнее скорости его первичного проектирования.

3 ЛК 3: Кодирование ПО (1)

Практические советы

1. Используйте идиоматичные для языка конструкции (например, `goto` — только в исключительных случаях, `return 0` — как стандарт).
2. **Программная единица (ПЕ):** любой функционально законченный набор действий.
3. Следует избегать излишней детализации ПЕ в программе (некоторые функции в одну строчку).
4. **DRY (Don't Repeat Yourself):** код, используемый более одного раза, должен быть в ПЕ. Противоположность — антипаттерн **WET** (We Enjoy Typing).
5. Обязательная проверка входных параметров на корректность.
6. **Чистота функций:** избегайте вывода сообщений (print/cout) внутри ПЕ. Используйте *коды возврата*.
7. Избегайте «магических чисел». Используйте именованные константы.
8. Глобальные переменные — не для счетчиков циклов или служебных нужд внутри ПЕ.
9. Одна переменная — одна роль. Не используйте ее для разных целей.
10. **Инкапсуляция:** скрывайте детали реализации за правильными абстракциями.
11. Всегда инициализируйте переменные при объявлении.
12. Обрабатывайте ветки `else` и `default` в `if/switch`.
13. Опасайтесь неявного приведения типов (`autocast`).
14. Не злоупотреблять `auto`.
15. **Принцип модульности:** не следует помещать весь код в один файл.
16. **Принцип единственной ответственности:** одна функция — одна задача.
17. не услышал FIXME
18. Используйте промежуточные переменные для сложных вычислений.

4 ЛК 4: Кодирование ПО (2). Документирование ПО

Продолжение ЛК 3

19. Не следует самостоятельно существующие в языке программирования функции и давать собственным функциям названия, похоже на существующие.
20. Не следует использовать программу с 5 и более параметрами.
21. Желательно использовать операторы множественного выбора (`switch`) вместо кучи конструкций из операторов бинарного выбора (`if`).
22. Что-то про Вложенные циклы (...) .
23. Оператор `goto` — зло. Причина становления кода спагетти-кодом.
24. Бесконечный цикл нужно использовать осознанно.
25. Использование `#define` для объявления констант (устаревшая практика). Макросы нельзя отлаживать – это главный минус.
26. Наблюдение: в большинстве случаев удается либо сократить число ПЕ, либо сделать их код компактнее, но не решить эти обе задачи (гнаться за этим часто не стоит).
27. Следует избегать зависимости от порядка вычислений.

Документирование ПО

1. Код без комментариев не имеет смысла (не поддается нормальному сопровождению).
2. Комментарии должны быть понятными.
3. Не следует вставлять комментарий внутри программного кода.
4. Комментарии не должны быть бессодержательными или излишне многословно. Совет: 1 комментарий в среднем на 7-10 строк кода.
5. Предпочтительнее, чтобы комментарии поясняли не что написано, а почему это написано именно так.
6. Для удобства восприятия следует выравнивать комментарии в тексте программы (улучшение визуализации).
7. Следует не забывать корректировать комментарии при изменении программного кода (ложные комментарии лучше, чем отсутствующие). Для напоминания используют комментарии вида `TODO`, `FIXME`, `XXX`.
8. Следует использовать стандартизованные отступы для обозначения уровня вложенности.

9. Следует отмечать конец крупного составного оператора/блока каким-либо комментарием.
10. На строке следует размещать одну операцию (x++; y++; в одной строке).
11. Код ПЕ должен охватываться взглядом (он должен вмещаться на странице и/или экране).
12. какой-то пункт пропустил.

5 ЛК 5: Документирование ПО (2). Советы для C/C++

Продолжение ЛК 4

13. Следует визуально разделать подпрограммы пустыми строками, звездами и т.д.
14. Следует использовать пустые строками.
15. Следует активно использовать пробелы и скобки для читабельности.
16. Части кода, относящиееся к одной задаче, располагаются рядом.
17. Следует использовать осмысленные мнемонические идентификаторы; избегать использования в качестве идентификаторов выдуманных аббревиатур, а также образования идентификаторов путём удаления букв или гласных.
18. Не надо переопределяит идентификаторы, которые уже и так препопределены.
19. Не следует использовать в различных местах для обозначения идентификаторов буквы различных регистров.
20. Не следует использовать слишком длинные идентификаторы для читабельности кода.
21. Не следует использовать идентификаторы (имена переменных, функций, классов) с орфографическими ошибками или с незначительными отклонениями в написании.
22. Использовать для конкретного языка программирования те или иные соглашения по именования (в C/C++, Python – snake case, в Pascal – pascal case, в Java, Go – camel case).
23. Существуют общепринятые короткие имена для переменных, которые понятны любому программисту без лишних слов. Например:
    - ◯ a, b, c: Обычно используются для коэффициентов в математических формулах (как в квадратном уравнении) или для обозначения границ интервалов.
    - ◯ p, q, ptr: Сокращения для указателей (pointers).
    - ◯ ch: Сокращение от character (символ).
    - ◯ s, str: Сокращения для string (строка).
24. Не следует начинать или заканчивать имена переменных двойным подчеркиванием.

Советы для C/C++

1. Не стоит писать всё только маленькими буквами без разделения (~~helloworldvariable~~ –> hello_world_variable).
2. Идентификаторы классов и структур — с большой буквы.
3. TBS – ??? FIXME
4. Присваивание – ??? FIXME
5. Использовать комбинированные операторы (+=, -=, *= и т.д.).
6. Не злоупотреблять `using namespace std`, а указывать пространство имен явно.
7. FIXME.

Стиль программирования: вывод

`Вывод: хороший стиль влияет на скорость, трудоемкость и качество разработчика. Влияет на коллективную разработку.`

6 ЛК 6: Разговоры о разном (1)

Определения

- **Операнд** — данные, над которыми мы совершаем действие (числа, переменные).
- **Операция** — само действие.
- **Оператор** — символ этого действия (например, `+`, `-`, `&`).

Операции делятся по **арности** (количеству операндов): унарные (один), бинарные (два) и тернарные (три).

Когда всё идёт не по плану

В языках вроде C++ программа не всегда работает предсказуемо. Есть два термина:

1. **Unspecified Behavior** (неуточненное) — результат зависит от того, какой у тебя компилятор и процессор. Например, размер типа `int`. Это нормально, но нужно иметь в виду.
2. **Undefined Behavior (UB)** (неопределенное) — полная анархия. Стандарт языка не обещает вообще ничего. Программа может выдать мусор, упасть или отформатировать жесткий диск.

Самые частые причины UB:

- Работа с переменной, которой не задали начальное значение.
- Переполнение типа данных (когда число не влезает в память).
- Деление на `0`.
- Попытка обратиться к пустому указателю.
- Выход за границы массива.

Тот самый пример из тетради

Смотрим на этот код:

```
int x = 5;
int y = 7;
int z = (x = y * 2) + (x = 1);
```

Мнение лектора: Здесь мы пытаемся изменить значение `x` дважды внутри одного выражения. Проблема в том, что оператор сложения `+` не гарантирует порядок вычисления скобок. Компилятор может сначала вычислить левую часть ($x = 14$), а потом правую ($x = 1$). А может наоборот! В итоге в `z` запишется непонятно что. Это классическое UB.

Битовая магия

Помимо обычной математики и логики (И, ИЛИ, НЕ), есть побитовые операции:

1. Побитовое И (`&`), ИЛИ (`|`), НЕ (`~`).
2. Исключающее ИЛИ (`^`) — тот самый XOR.
3. Сдвиги битов (`«`, `»`).

Битовая маска — это изящный способ изменить только конкретные биты в байте, не сломав остальные. Чаще всего делается через операцию `&` с заранее заготовленным числом.

7 ЛК 7: Разговоры о разном (2)

Побитовые операции

1. XOR ($0 \oplus 0 = 1 \oplus 1 = 0$, $1 \oplus 0 = 0 \oplus 1 = 1$): коммутативен ($X \oplus Y = Y \oplus X$) и является обратимым ($X \oplus Y \oplus X = Y$).
2. Битовый сдвиг влево на n это 2^n, вправо – 2^{-n}.

Сегментация данных

1. Сегмент исходного кода (CS).
2. Дата сегмент (DS).
3. Стек (stack).
4. Куча (heap).

Память

1. Выделение памяти (new).
2. Освобождение памяти (delete).

Работа с указателями и кучей (Heap)

1. **Выделение переменной в куче:** Оператор `new` запрашивает блок памяти в куче и возвращает адрес (указатель) на начало этого блока.

```
int* ptr = new int(42); // Выделили память под int и сразу записали 42
```

2. **Разыменование (разадресация):** Получение доступа к самому значению, которое лежит по адресу памяти. Выполняется с помощью оператора `*`.

```
int value = *ptr; // Читаем значение по адресу (получим 42)
*ptr = 100;       // Записываем новое значение 100 по этому адресу
```

3. **Динамические массивы:** В отличие от статических массивов на стеке, размер динамического массива в куче можно задавать переменной прямо во время выполнения программы.

```
int n = 10;
int* arr = new int[n]; // Выделили память под 10 элементов типа int
arr[0] = 5;            // Работаем как с обычным массивом
```

```
delete[] arr; // ВАЖНО: массивы удаляются с помощью delete[]
```

Утечки памяти (Memory Leaks)

1. **Суть проблемы:** Ситуация, когда память в куче была выделена через `new`, но программист забыл вызвать `delete`, а указатель на эту память был потерян (например, переменная указателя вышла из области видимости).

```
void badFunction() {
    int* ptr = new int(10);
    // Забыли написать delete ptr;
    // При выходе из функции переменная ptr (лежащая на стеке)
    // уничтожится, а выделенные байты в куче навсегда останутся занятыми.
}
```

Арифметика указателей

1. **Сдвиг адреса:** К указателю можно прибавлять или вычитать целые числа. Прибавление единицы сдвигает адрес не на 1 байт, а ровно на **размер типа данных**, на который он указывает (например, на 4 байта для `int`).

```
int* arr = new int[3]{10, 20, 30};
int val = *(arr + 1); // Сдвинулись на 1 элемент вперед (получим 20)
// arr[1] --- это синтаксический сахар для *(arr + 1)
```

2. **Разность указателей:** Можно вычесть один указатель из другого (если они указывают на элементы одного и того же массива). Результатом будет не разница в байтах, а количество элементов между ними.
3. **Недопустимые операции:** Указатели нельзя складывать друг с другом, умножать или делить.

Остаток от деления (%) с отрицательными числами

1. **Главное правило** Знак результата операции `a % b` всегда совпадает со знаком **делимого** (левого операнда `a`). Знак делителя (`b`) вообще ни на что не влияет.
2. **Примеры:**

```
 5 %  3 ==  2  // Классика
```

```
-5 %  3 == -2  // Знак берется от делимого (-5)
 5 % -3 ==  2  // Знак делителя (-3) игнорируется
-5 % -3 == -2  // Знак берется от делимого (-5)
```

Инкремент (++) и декремент (–)

Эти унарные операторы увеличивают или уменьшают значение переменной на единицу. Разница заключается в том, **когда** именно происходит вычисление внутри сложного выражения.

1. **Префиксная форма (`++x`, `-x`):** Сначала переменная изменяется, а затем её новое значение возвращается и используется в выражении.

   ```
   int a = 5;
   int b = ++a; // Сначала 'a' стало 6, потом 'b' получило значение 6.
   // Итог: a = 6, b = 6
   ```

2. **Постфиксная форма (`x++`, `x-`):** Сначала возвращается *текущее* (старое) значение переменной для использования в выражении, и только потом сама переменная изменяется.

   ```
   int x = 5;
   int y = x++; // 'y' получает старое значение 5, затем 'x' становится 6.
   // Итог: x = 6, y = 5
   ```

Логические операторы: приоритеты и «ленивость»

1. **Приоритеты (порядок выполнения):** Если в выражении нет скобок, компилятор расставляет приоритеты от высшего к низшему:

   - `!` (Логическое НЕ) — выполняется самым первым (как унарный минус).
   - `&&` (Логическое И) — приоритет выше, чем у ИЛИ (аналог умножения).
   - `||` (Логическое ИЛИ) — выполняется последним (аналог сложения).

   Мнение лектора: Важно помнить: операторы сравнения (`==`, `>`, `<`) имеют приоритет ВЫШЕ, чем `&&` и `||`. Поэтому выражение `if (a == b && c > d)` сработает правильно и без дополнительных скобок. А в выражении `A || B && C` сначала выполнится `B && C`.

2. **Ленивое вычисление (Сокращенная схема выполнения логических операций):** Компилятор читает логические выражения строго слева направо.

Как только итоговый результат становится математически неизбежным, вычисление **мгновенно прекращается**.

- Для `&&` (И): Если первая часть `false`, всё выражение будет `false`. Правая часть **не вычисляется**.
- Для `||` (ИЛИ): Если первая часть `true`, всё выражение будет `true`. Правая часть **не вычисляется**.

3. **Как это спасает от ошибок (защита от UB):** Благодаря ленивости мы можем объединять проверку безопасности и само действие в один `if`.

```
int* ptr = nullptr;

// Если ptr равен nullptr, левая часть вернет false.
// Из-за && правая часть (*ptr == 5) просто проигнорируется!
// Программа спасена от UB (разадресации нулевого указателя).
if (ptr != nullptr && *ptr == 5) {
    // Делаем что-то...
}

int x = 0;
// Защита от краша из-за деления на ноль в одну строку:
if (x != 0 && (100 / x) > 5) {
    // Делаем что-то...
}
```